

Laboratorio 3

Dopo 501 2026-2

Escuela Colombiana de Ingeniería Julio Garavito

**Hecho por: Juan David Espitia - Cristian Camilo
Salamanca**

Profesor: Maria Irma Diaz Rozo

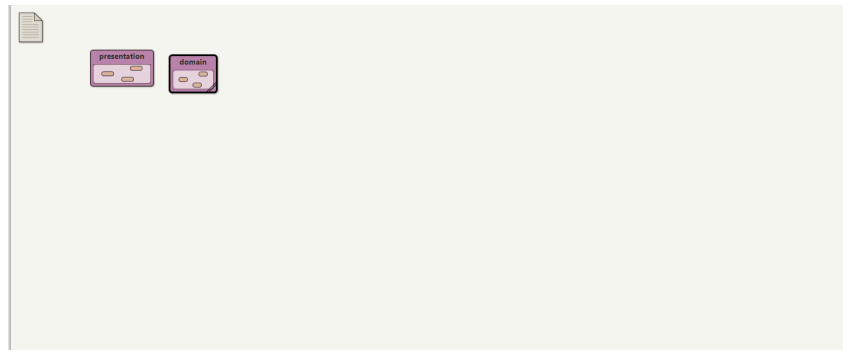
Parte I

A. Revisando línea base [En lab03.pdf]

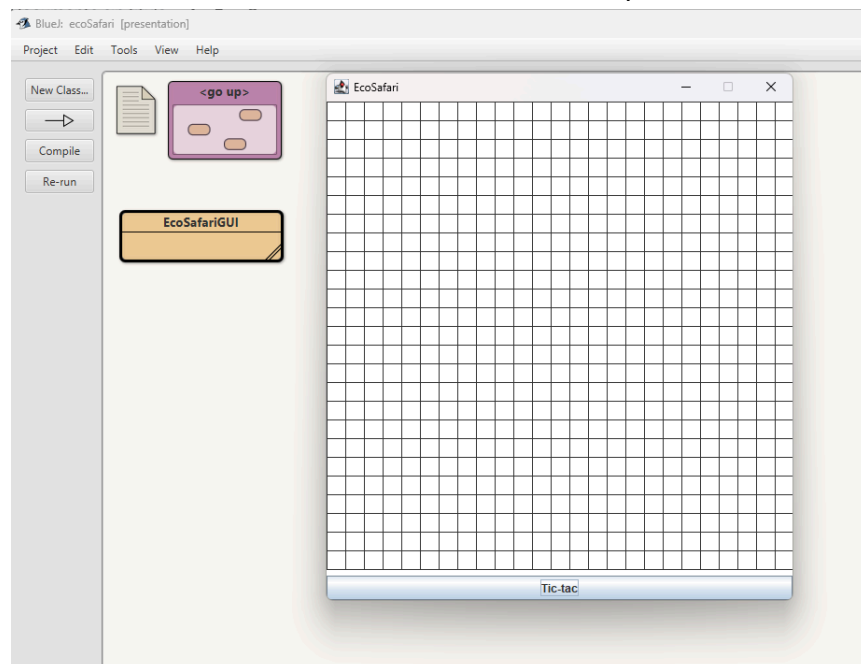
1. En el directorio descarguen los archivos contenidos de safari.zip. Revisen el código: (a) ¿Cuántos paquetes tiene? (b) ¿Cuál es el propósito del paquete presentación? (c) ¿Cuál es el propósito del paquete dominio?

R/

- a) Cuenta con dos paquetes, el paquete de domain y presentación. Y en la interfaz de BlueJ aparecen de la siguiente forma:

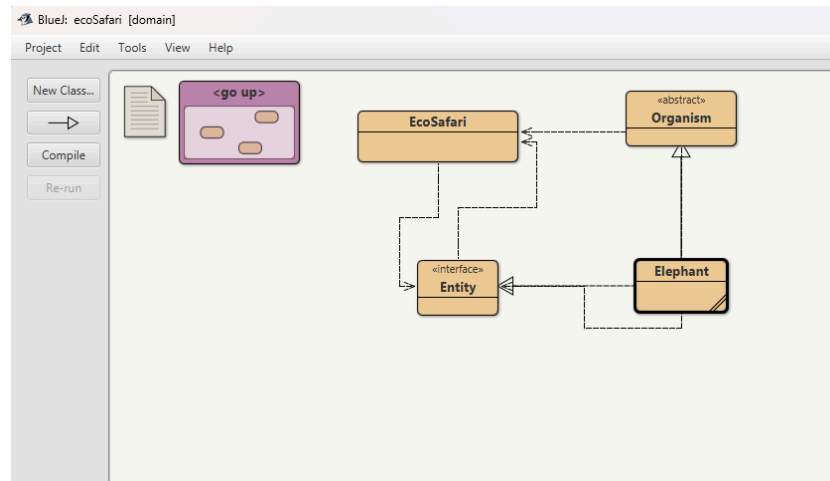


- b) Es el paquete encargado de almacenar, la clases de interfaz grafica con un JFrame para el usuario. En ella se encuentra la clase de EcoSafariGUI, esta clase está encargada de construir por el usuario una cuadrícula 2D donde cada casilla mide 20 píxeles.



- c) Mientras que la parte de présentation se encarga de la parte gráfica, domain se encarga de toda la parte lógica, en que se se calcula detrás del proceso. Y que construye todo lo que necesita el programa para responder satisfactoriamente por las funciones solicitadas. En ella se encuentran las clases de EcoSafari; Organism, Entity,

Elephant.



2. Revisen el paquete de dominio, (a) ¿Cuáles son los diferentes tipos de componentes de este paquete? (b) ¿Qué implica cada uno de estos tipos de componentes?

R/

- a) En una primera instancia, se ve que el paquete domain contiene distintas clases como lo son EcoSafari, Organism, Entity, Elephant. En cuanto a tipos de componentes se sabe que la clase Entity es una interfaz y que la clase Organism es una clase abstracta. Además que la clase Elephant hereda de Entity y Organism.

b)

- Clase abstracta: Clase abstracta que actúa como molde los atributos y comportamientos comunes de los seres vivos.
- Interfaz: Interfaz que define el contrato o comportamiento base para cualquier clase.
- Relación de herencia: Elephant tiene relación hacia Organism y hacia Entity lo que significa que un elefante es un tipo de organismo. Entonces este puede heredar comportamientos y atributos de sus clases padres.

3. Revisen el paquete de presentación, (a) ¿Cuántos componentes tiene? (b) ¿Cuántos métodos públicos propios (no heredados) ofrece?

R/

- a) El paquete de presentación cuenta con una sola clase, que es EcoSafariGUI y dentro de esta clase se encuentra la clase heredada de Photo EcoSafari.

b) El metodo publico que este puede ofrecer, es el de gettheEcoSafari

4. Para ejecutar un programa en java, (a) ¿Qué método se debe ejecutar? (b) ¿En qué clase se encuentra? Deben ejecutar la aplicación java, no crear un objeto como lo veníamos haciendo

R/

- a) Para ejecutar el programa se debe ejecutar dentro del paquete de presentación

- b) Dentro de la clase de EcoSafariGUI, se debe ejecutar el método de main, en donde llamará a diferentes clases y métodos para implementar un JFrame donde va a estar la cuadrícula, donde se verá la simulación
- 5. Ejecuten el programa. (a) ¿Qué funcionalidades ofrece? (b) ¿Qué hace actualmente? (c) ¿Por qué?
 - a) La funcionalidades que ofrece a la hora de la ejecución es la de la visualización de la cuadrícula, de 20 x 20, un botón Tic-Tac que dentro del código está encargado de volver pintar la cuadrícula.
 - b) Actualmente el botón vuelve a pintar a cuadrícula, pero como no están implementados los sujetos que pueden interactuar con esta cuadrícula, está siempre va a quedar inicializada o queda en blanco

B. Realizando ingeniería inversa. Arquitectura General [En lab03.pdf safari.astah]

1. Consulten el significado de las palabras package e import de java. (a) ¿Qué es un paquete? (b) ¿Para qué sirve? (c) ¿Para qué se importa? (d) Expliquen su uso en este programa.
R/
Package: Hace referencia a una carpeta en donde se almacenen las clases a necesidad del proyecto.
 - a) Para organizar, separar y organizar por tareas y objetivos del proyecto por lo general en proyectos se organiza por front(interfaz usuario) y dack(lógica de programa)
 - b) Se importa con el fin de
 - c) se importa con el fin de indicar al compilador dónde encontrar las clases que están fuera del paquete actual. Se trata de mecanismos que nos ayudan a organizar clases de Software y otros elementos del proyecto de esta manera clasificar aspectos generales e implementar de manera ordenada las clases.
 - d) En el programa aparecen dos paquetes, el de présentation que se encarga de la presentación e interfaz gráfica de usuario del programa. Por otro lado, domain se encarga de almacenar las funciones de lógica y backend.
2. Revisen el contenido del directorio de trabajo y sus subdirectorios. Describan su contenido. (a) ¿Qué coincidencia hay entre paquetes y directorios?
 - a)
3. Adicionen al diseño la arquitectura general con un diagrama de paquetes en el que se presente los paquetes y las relaciones entre ellos. Consulte la referencia en Moodle. En astah, crear un diagrama de clases (cambiar el nombre por Package Diagram0)

C. C Realizando ingeniería inversa. Arquitectura detallada [En lab03.pdf safari.astah]

1. Hacían falta las clases que representan los diferentes tipos de organismos del dominio, principalmente Animal y Plant, junto con sus respectivas especializaciones concretas según las especies definidas en el problema. También era necesario completar las relaciones de herencia entre Entity, Organism, Animal y Plant, y desarrollar el comportamiento de EcoSafari.ticTac(), encargado de ejecutar las acciones tic() y tac() de las entidades.
2. Hay dos clases, EcoSafariGUI y PhotoEcoSafari, porque cada una tiene una responsabilidad diferente dentro de la presentación: EcoSafariGUI administra la ventana y la interacción con el usuario, mientras que PhotoEcoSafari se encarga de dibujar gráficamente el EcoSafari. Existe un solo archivo .java porque PhotoEcoSafari está declarada como una clase interna dentro de EcoSafariGUI, por lo que ambas clases pueden estar en el mismo archivo EcoSafariGUI.java.
- 3 Hay dos clases, EcoSafariGUI y PhotoEcoSafari, porque cada una tiene una responsabilidad diferente dentro del paquete de presentación. EcoSafariGUI se encarga de administrar la ventana, los componentes de la interfaz y la interacción con el usuario, mientras que PhotoEcoSafari se encarga de representar gráficamente el estado del EcoSafari.

Existe un solo archivo .java porque PhotoEcoSafari está declarada como una clase interna dentro de EcoSafariGUI. Por esta razón, ambas clases pueden estar contenidas en el mismo archivo EcoSafariGUI.java.

Parte II Desarrollando el Proyecto

A. Ciclo uno: Iniciando con leones:

1. **Estudie la clase EcoSafari . (a) ¿Qué tipo de colección usa para albergar cosas? (b)¿Puede tener leones ? (c) ¿Por qué?**
 - a) Inicialmente la clase EcoSafari cuenta con solo una única colección, en donde se usa un arreglo doble asociado a las celdas que manejan el simulador a realizar.
 - b) Inicialmente no, ya que no hay una clase específica para los leones inicialmente, sin embargo, en cuanto se implemente, será posible que los objetos de dicha clase puedan ser albergados en esta colección.
 - c) viendo el ejemplo que está presente de elefante, es posible ver como este hereda de Organism y a su vez implementa la interfaz entity, permitiendo ser albergado en la colección, por lo que es logico que para la clase León será igual
2. **Estudie el código asociado a la clase Elephant , (a) ¿de qué color es? (b) ¿qué forma usa para pintarse? (c) ¿cómo se mueve ? (d) ¿qué componentes definen la clase Elephant ? Justifique sus respuestas.**
 - a. Este depende de un atributo y método heredados de su clase padre Organismo, el cual es energy y getEnergy(), en donde si este atributo es mayor a 80, es DARK_GRAY, de lo contrario es LIGHT_GRAY

- b. El único método respecto al color es el de getColor(), el cual es el mismo que se indica en el inciso, que hace una toma de decisión acerca del color del Elephant .
- c. Se mueve haciendo uso de los distintos métodos que ofrece la interfaz Entity, ya que esta es responsable del método move, el cual es usado en el método tic, el cual también viene de la interfaz, la diferencia es que move es un método de tipo default en esta implementación.
- d. Los componentes que la componen, es que hereda de Organism y hereda de Entity.

3. Elephant por ser un Organism , (a) ¿qué atributos tiene? (b)¿qué puede hacer (métodos)? (c) ¿qué decide hacer distinto (sobre-escritura)? (d) ¿qué no puede hacer distinto (métodos finales)? (e) ¿qué debe aprender a hacer (métodos abstractos)? Justifique sus respuestas.

- a. Siendo propios cuenta con atributos: hábitat de tipo EcoSafari, otro booleano llamado hasActed. Mientras que heredados cuenta con otros que son llamados: Energy el cual es un dato primitivo int.
- b. Si se observa únicamente los métodos propios y heredados de Organism, cuenta con getColor, getHabitat, shape, tic, tac, además de los propios métodos de la clase padre, los cuales son, dos changeEnergy, un getEnergy, y un isOrganism, los cuales, en su totalidad, son final, por eso ninguno tiene la necesidad y tampoco puede ser sobreescrito.
- c. implementa métodos propios, sin embargo no sobreescibe ninguno de su clase padre ya que esta no se lo permite.
- d. Todos los métodos que se heredan de Organism no son sobreescribibles ya que son declarados final.
- e. no debe aprender nada obligatoriamente, pues, no cuenta con metodos abstractos en su clase padre.

4. Por comportarse como un Entity , (a) ¿qué debe hacer? (b) ¿qué decide hacer distinto? (c) ¿qué no puede hacer distinto? (d) ¿qué debe aprender a hacer? Justifique sus respuestas.

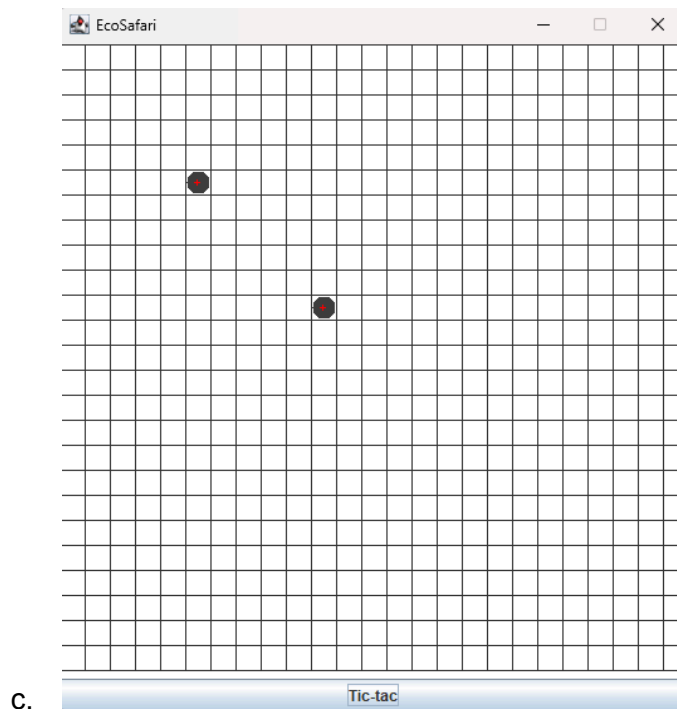
- a. Tiene un conjunto de comportamientos, tic, tac, move, shape, getColor, isOrganism, getHabitat y disappear.
- b. los que podría hacer distinto con sobreescritura, son los default, en donde se pueden redefinir o sobreescribir, en este caso no usa sobreescritura.
- c. La interfaz no cuenta con ningún final que prohíba la sobreescritura de los cuales ya están definidos.
- d. debe aprender con los métodos que solo tienen su firma sin default y los que son declarados como abstractos, tales como getColor y tic.

5. De acuerdo a lo anterior un Elephant , (a) ¿Cómo actúa?

- a. Actúa de manera robusta, por un lado tiene todos los comportamientos y atributos de su clase padre, en donde además firma un contrato para cumplir con los comportamientos de Entity, en

donde por un lado ser un Organism le da las características propias como organismo, mientras que lo que le aporta Entity, es a las acciones propias y básicas que tiene como entidad.

6. **Ahora vamos a crear dos Elefantes en diferentes posiciones (5,5) (10,10) , llámelos dumbos y babar , usando el método someEntities . Ejecuten el programa, (a) ¿Qué pasa con los leones ? (b)¿Por qué? (c) Capturen una pantalla significativa.**
- a. Los elefantes aparecen en la cuadrícula del programa en sus posiciones correspondientes.
 - b. al inicializar el programa se inicia un EcoSafari que a su vez utiliza a someEntities que es quien inicializa a los elefantes.



7. **Diseñen, construyan y prueben el método llamado ticTac() de la clase EcoSafari .**

R// Teniendo en cuenta que el tic-tac determina cómo avanza un paso de tiempo en el simulador, lo correcto es hacer tic para todas las entidades y luego el tac para identificar que acabó, y como la estructura está dada por un arreglo doble de cells de tipo Entity, se desarrollo el siguiente código:

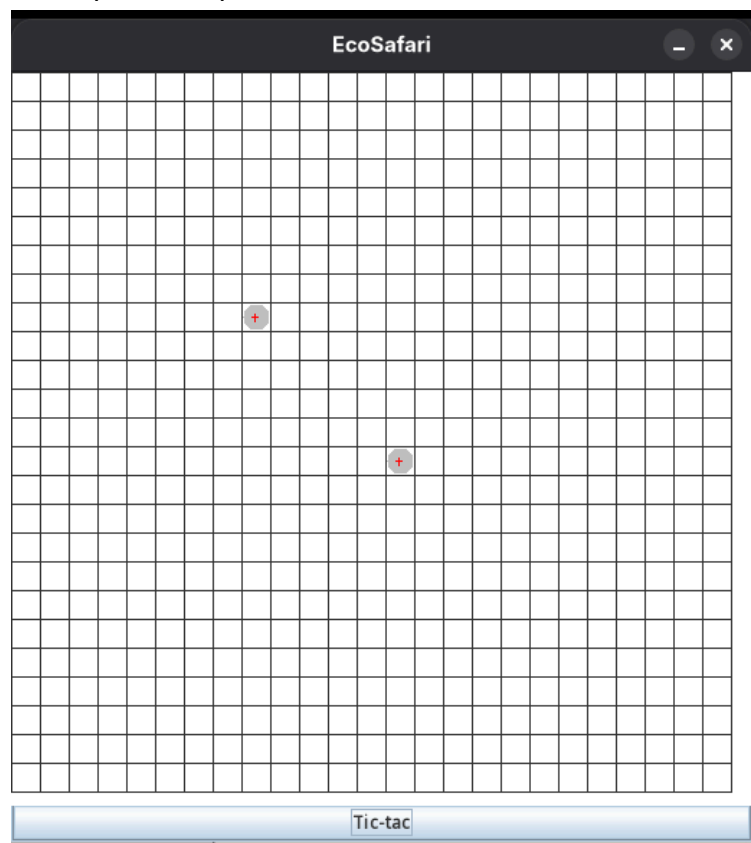
```

/**
 * Advances the simulation by one time step
 */
//First, all entities execute their tic() action
//Then, all entities execute their tac() actions
public void ticTac(){
    for(int i = 0; i < cells.length; i++){
        for(int j = 0; j < cells[i].length; j++){
            if(cells[i][j] != null){
                cells[i][j].tic();
            }
        }
    }

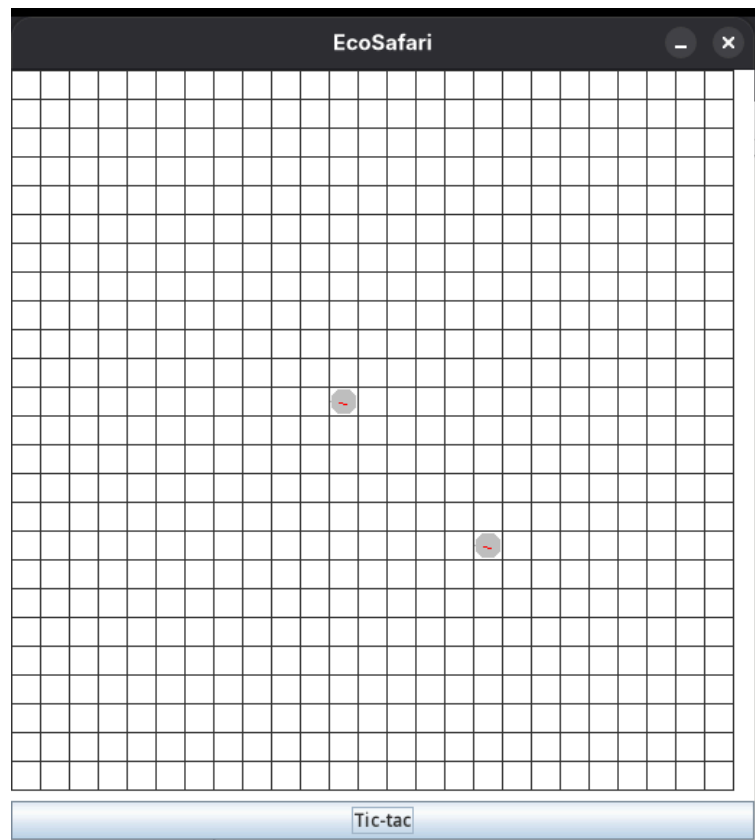
    for(int i = 0; i < cells.length; i++){
        for(int j = 0; j < cells[i].length; j++){
            if(cells[i][j] != null){
                cells[i][j].tac();
            }
        }
    }
}
}

```

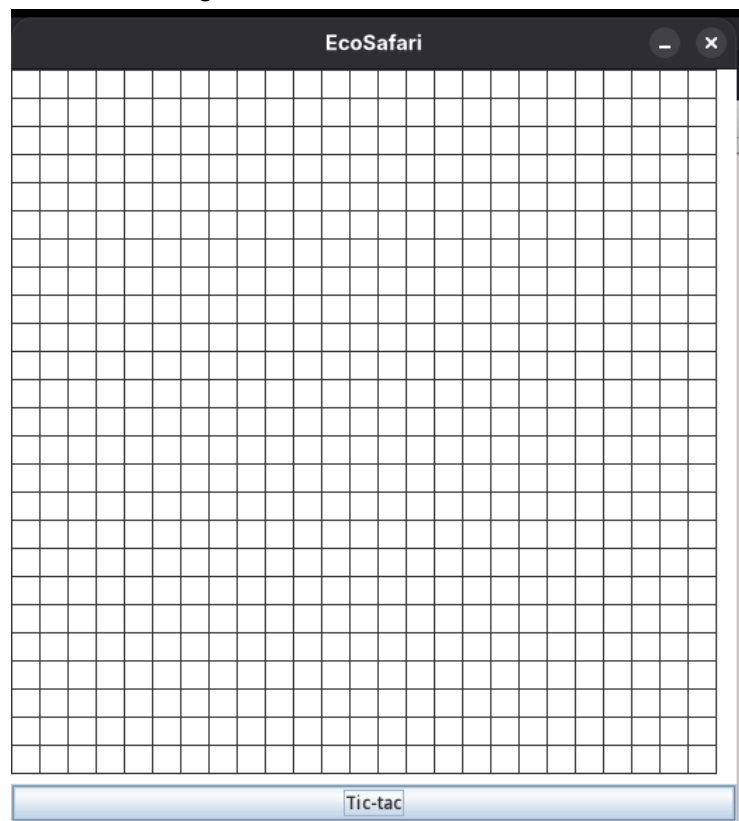
8. a) ¿Cómo quedarían dumbó y babar después de 3, 5, 10 tic-tac? Ejecuten el programa. Capturen pantallas significativas en los momentos correspondientes. (b) ¿Es correcto?
- a. Con 3 pasos se puede ver un cambio de color:



Con 5 pasos se puede ver un cambio de posición normal pero no ocurre un cambio significativo, sin embargo, con 6, se obtiene un cambio en el símbolo en el interior de la representación del elefante, en donde pasa de tener un símbolo “+”, a tener un “~”:



Para la siguiente toma que son 10 pasos se puede observar como desaparecen y la cuadrícula queda como cuando no se habían inicializado ningún elefante:



- b. Si, todo esto es correcto ya que sigue los lineamientos impuestos en un inicio, en donde cada paso desgasta la energía del elefante, lo cual lo hace cambiar de color, así mismo con el símbolo que contiene, y para el final si se queda sin energía se puede concluir que muere o desaparece.

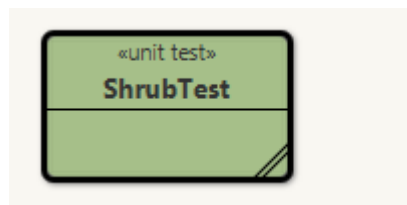
B. Ciclo 2. Incluyendo arbustos

1.

a) Los métodos que se tienen que hacer overriding o sobreescritura son los métodos enredados de la clase abstracta Organismo y la implementados desde la interfaz de Entity.

- Principalmente es necesario sobrecribir el método getColor() para la gestión de ese envejecimiento del arbusto cuando el arbusto tiene energía mayor a 60 mantiene el color verde y por el contrario cuando son menores a 60 el color es amarilloso (en otras palabras envejece al cuarto tic).
- El método Tic() para la lógica del ciclo donde se deba actualizar el estado arbusto, descontando 10 de energía en cada ciclo tic. también evaluar si es joven o a llegado a el segundo tic para desplazar y cuando llegue a 0 la energía aplicar el método público de disappear().
- En el Tac() se sobrecribe la fase de reseteo del turno, cambiando el estado de si esta actuando sea falso para habilitar el siguiente ciclo.
- El metodo de canBeEaten() para designarle el depredador principal el cual seria el elefante
- y el metodo de type() Identifica del tipo de entidad retornando la cadena de texto "Shrub"

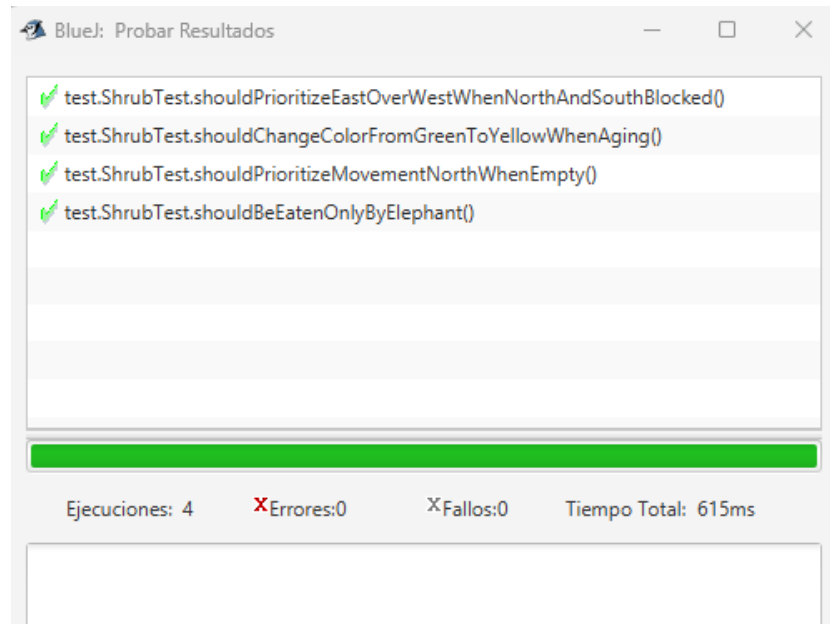
2.



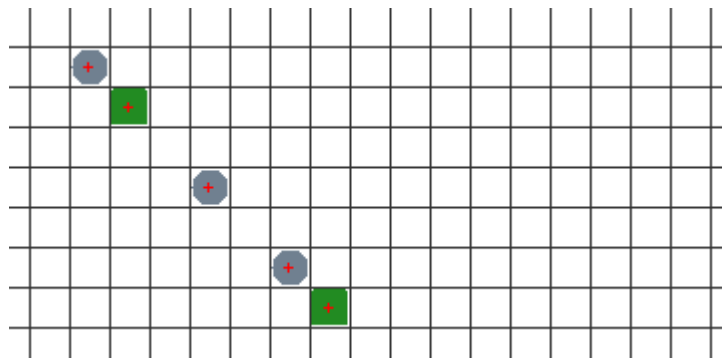
En la clase de Shrub Test, Cuenta con cuatro test.

1. la primera prueba consiste en verificar la regla de cambio de color segun el envejecimietno(nivel de energia) de cada arbusto.
Crear un arbusto con 100 de energia y comprueba que nace de color verde, se reduce la energia en 40 unidades y finalmente se valida el color amarillo
2. En la segunda prueba consiste en validar la primera prioridad de movimiento del arbusto(mover norte)
Valida la ubicacion inicial en (10,10) y posteriormente(9,10)
3. Confirmar que la prioridad Este prevalece sobre el Oeste cuando las casillas de Norte y Sur están desbloqueadas.
Coloca un arbusto en (10,10) y bloquea con bloques de tierra en (9,10) en (11,10)reduce al 80 la energía y verifica que el arbusto de desplace a (10,11)
4. Probar la regla de interacción con depredadores

Se instancia un elefante un león y una cebra, evalúa si canBeEaten con cada animal, confirma que la función solo retorna true para el elefante y false para el león



3. Los arbustos son los cuadrados de color verde oscuro



C. Ciclo 3. Adicionando tormentas

1.

a) No, a nivel de estructura de la cuadrícula ni de sus métodos principales. Mínimamente solo se modifica el método de `someEntities()` para instanciar e incluir las tormentas, y se completa la implementación del método `ticTac()`, para lograr ejecutar el ciclo de simulación si aún no estaba programado.

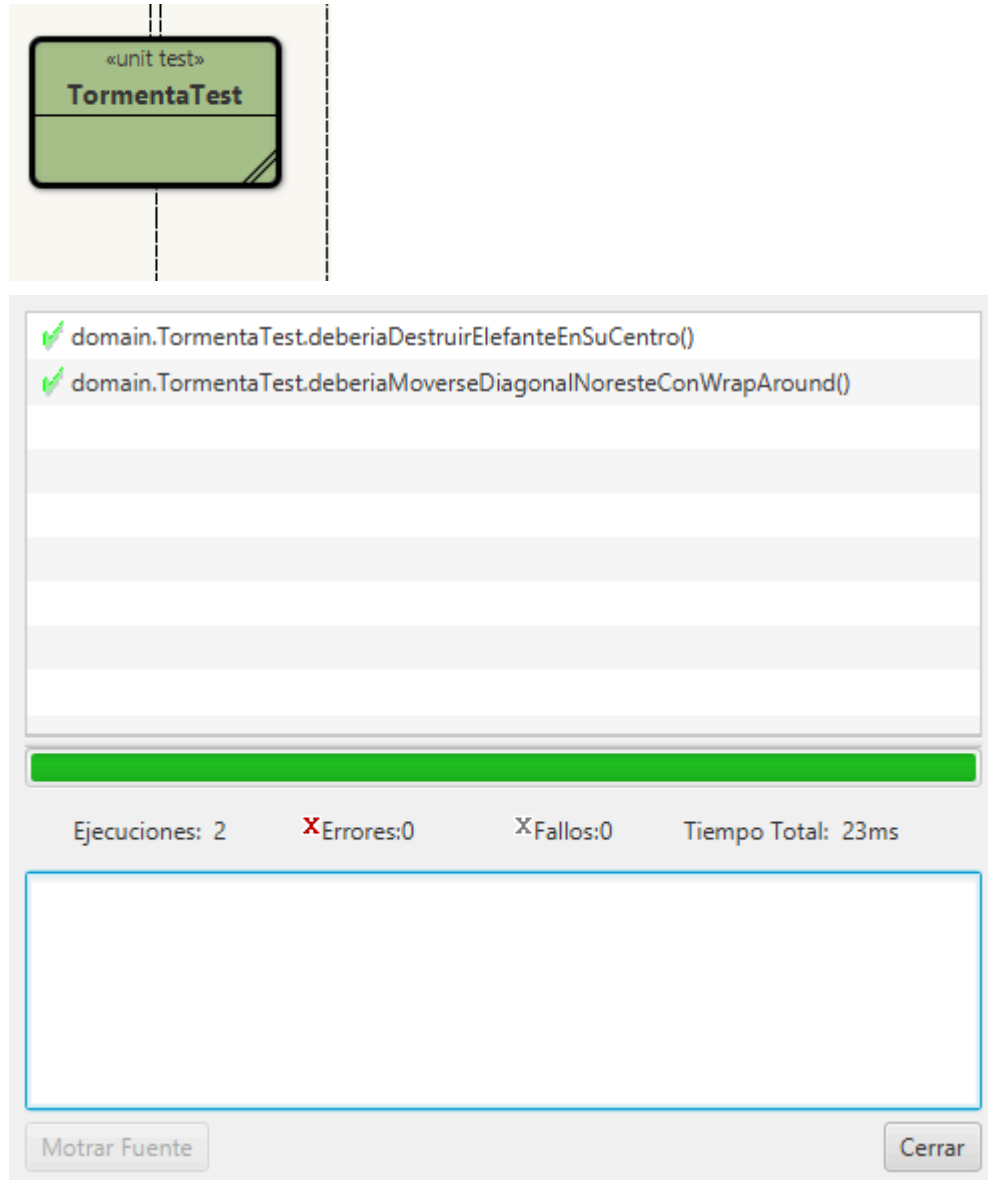
b) Principalmente de por los principios de Open/Closed y el Polimorfismo. La clase `EcoSafari` administra la matriz. Dado que los métodos de esta clase interaccionan con la parte de interfaz del paquete de presentación, no sería necesario modificar la estructura interna del hábitat para que acepten los nuevos tipos de entidades.

c) Si, la clase `Tormenta` se debe implementar la interfaz `Entity`

d) Ya que la tormenta sería un elemento que afecta directamente a el hábitat de `EcoSafari`. Esta ocupa casillas dentro de la cuadrícula, reacciona a el

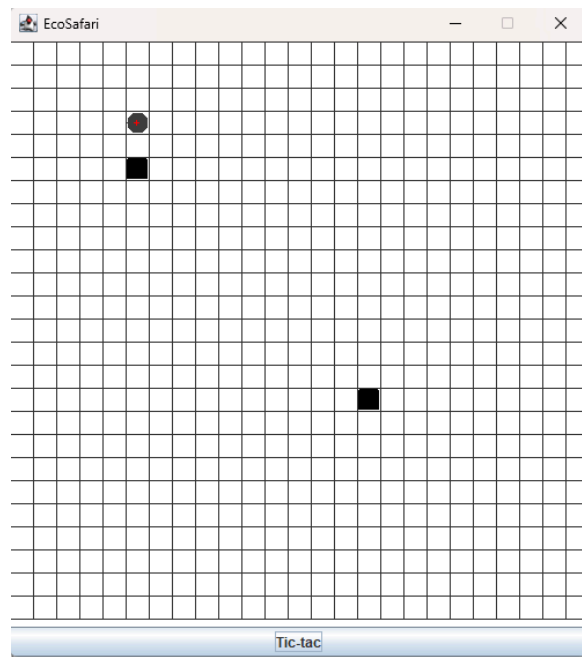
avance del tiempo ejecutando acciones en sus fases de tic() y tac(), y se debe mostrar mediante la interfaz grafica de EcoSafariGU.

2.



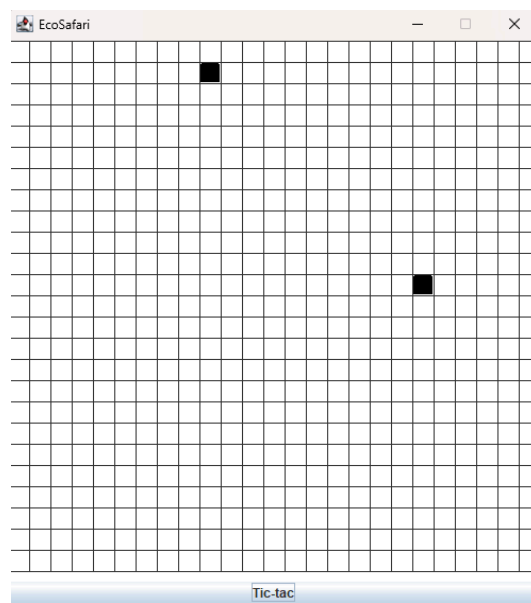
3.

a) Para determinar el movimiento de las tormentas solo basta con aplicar el desplazamiento en diagonal hacia el Noreste con Primera

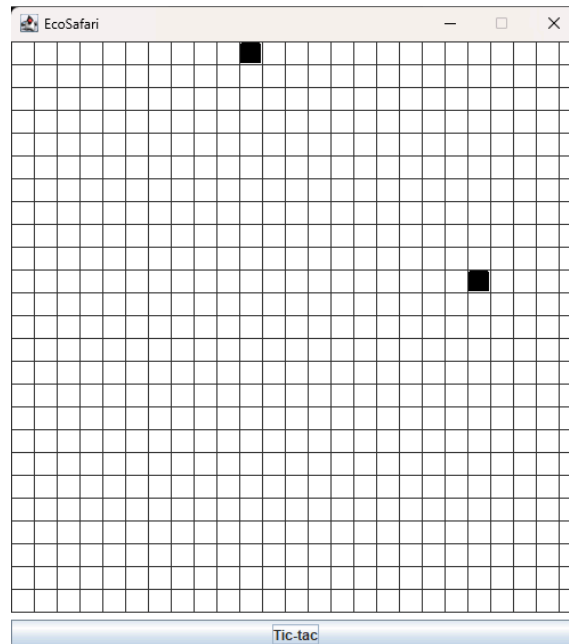


4.

Cuarta



b) Quinta



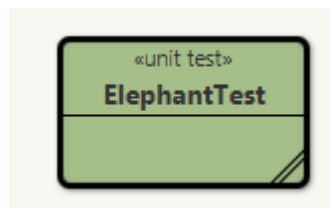
Si es correcta ya que la dirección y desplazamiento de las tormentas avanzan una casilla hacia arriba y una casilla a la derecha de manera constante en cada ciclo. Además de que la destrucción de las entidades usando disappear en cuando una tormenta llega a una casilla ocupada por una entidad

D. Ciclo 4. Proponiendo y diseñando : Nuevo elefante

1.

```
public void someEntities() {
    Elephant dumbo = new Elephant(this, 5, 5);
    Elephant babar = new Elephant(this, 10, 10);
}
```

2.

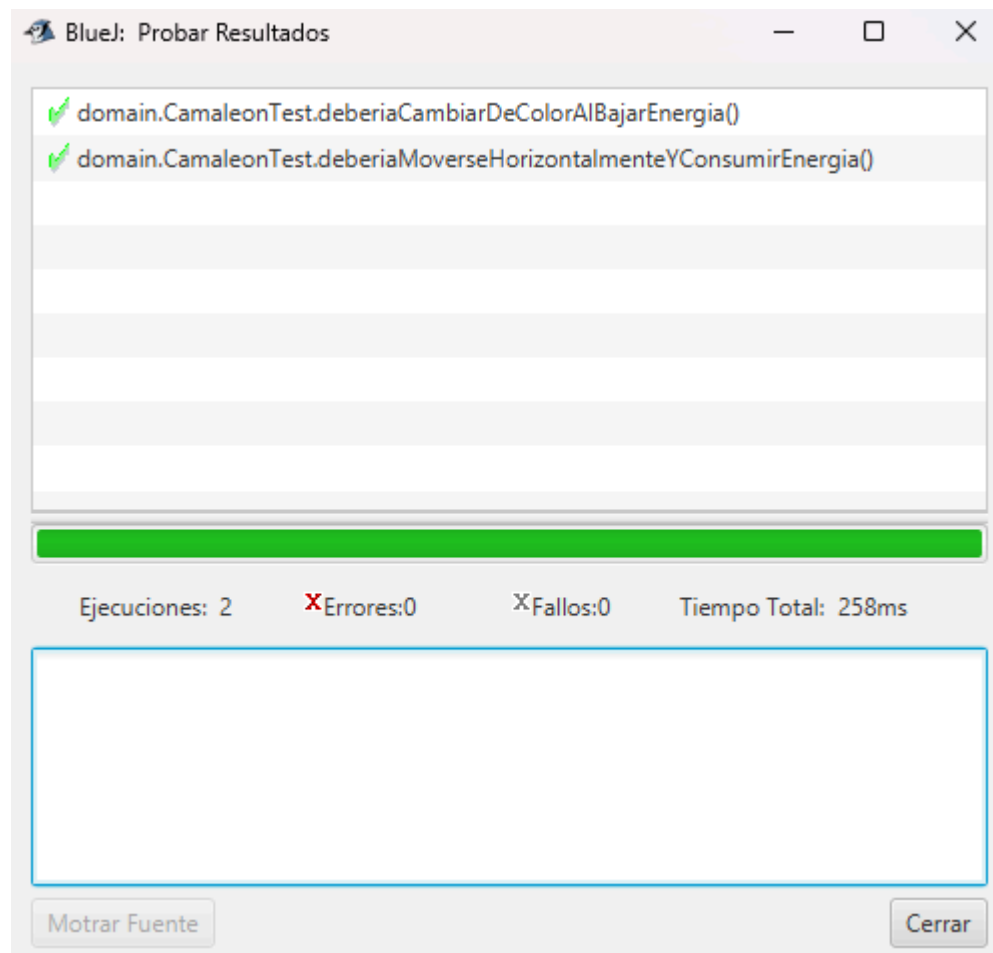


En esta clase se encuentran las pruebas. Cada uno de los test consisten en:

1. Verificar el patrón de movimiento diagonal del elefante y gastos de energía por turno.
2. Validar que el elefante se elimina automáticamente del tablero al agotar su energía
3. Confirmar la interacción predatoria con los arbustos y recuperación de esa energía
4. Probar el cambio de color dinamico del elefante cuando su energia es baja

E. Ciclo 5. Proponiendo y diseñando : Nueva entidad

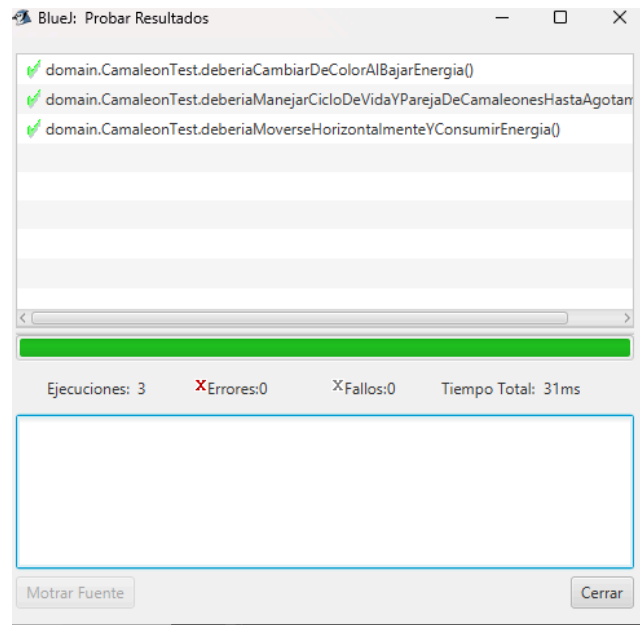
1. Proponemos una nueva entidad la cual se llama Camaleon, el cual representa un organismo vivo que cambia el de color segun el nivel de energia y se desplaza de forma horizontal. Va a ser de forma redonda. El color va a variar dependiendo de la energia, si esta es mayor o igual a 50, se muestra de color verde. Si su energia cae por debajo de 50, cambia a amarillo. en cada ciclo tic va a avanzar una casilla hacia la derecha y consumir 5 niveles de energía, si esta llega a cero se va a implementar el método disappear()



2.

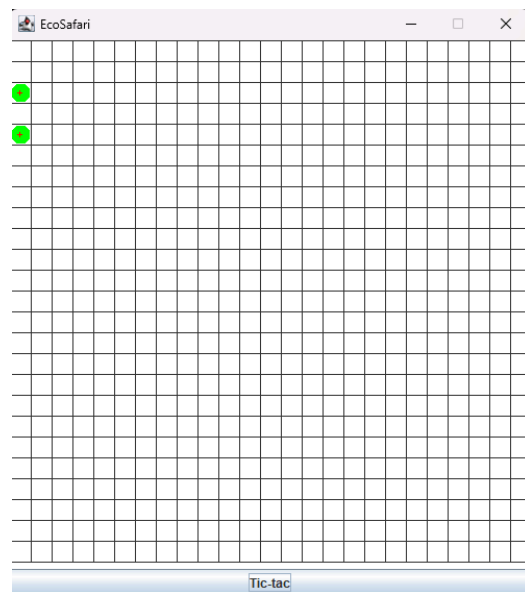
- a) Queremos verificar el ciclo de vida completo para una pareja de camaleones creados con el nombre de cada uno de nosotros (c1= Juan, c2 = Cristian). Se espera que se desplacen los dos simultáneamente por todo el tablero hasta que los dos desaparezcan, tras que su energía ya no exista después de varios ciclos tic.

b)

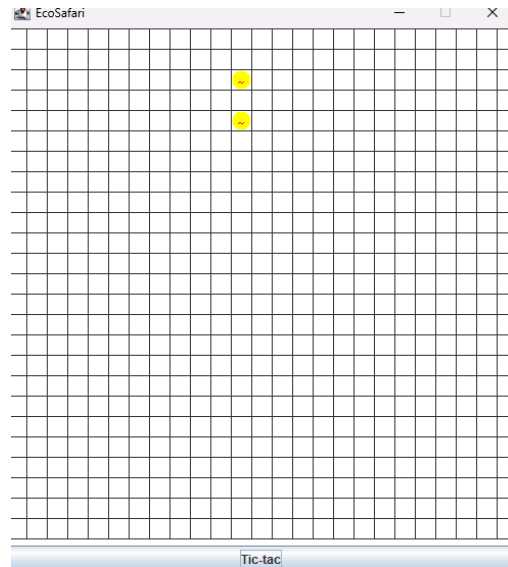


c)

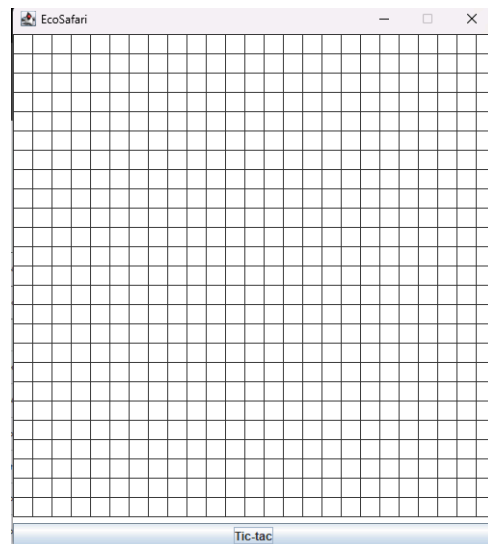
inicio:



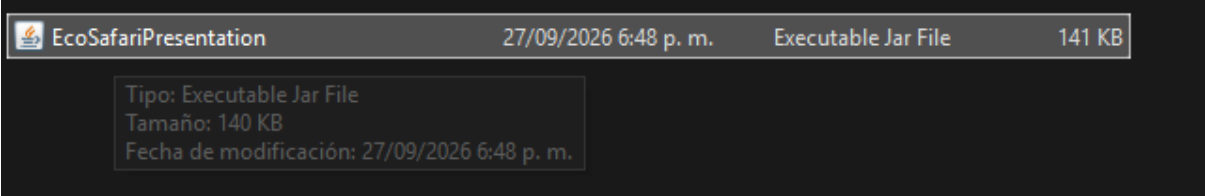
11 iteraciones de tic tac



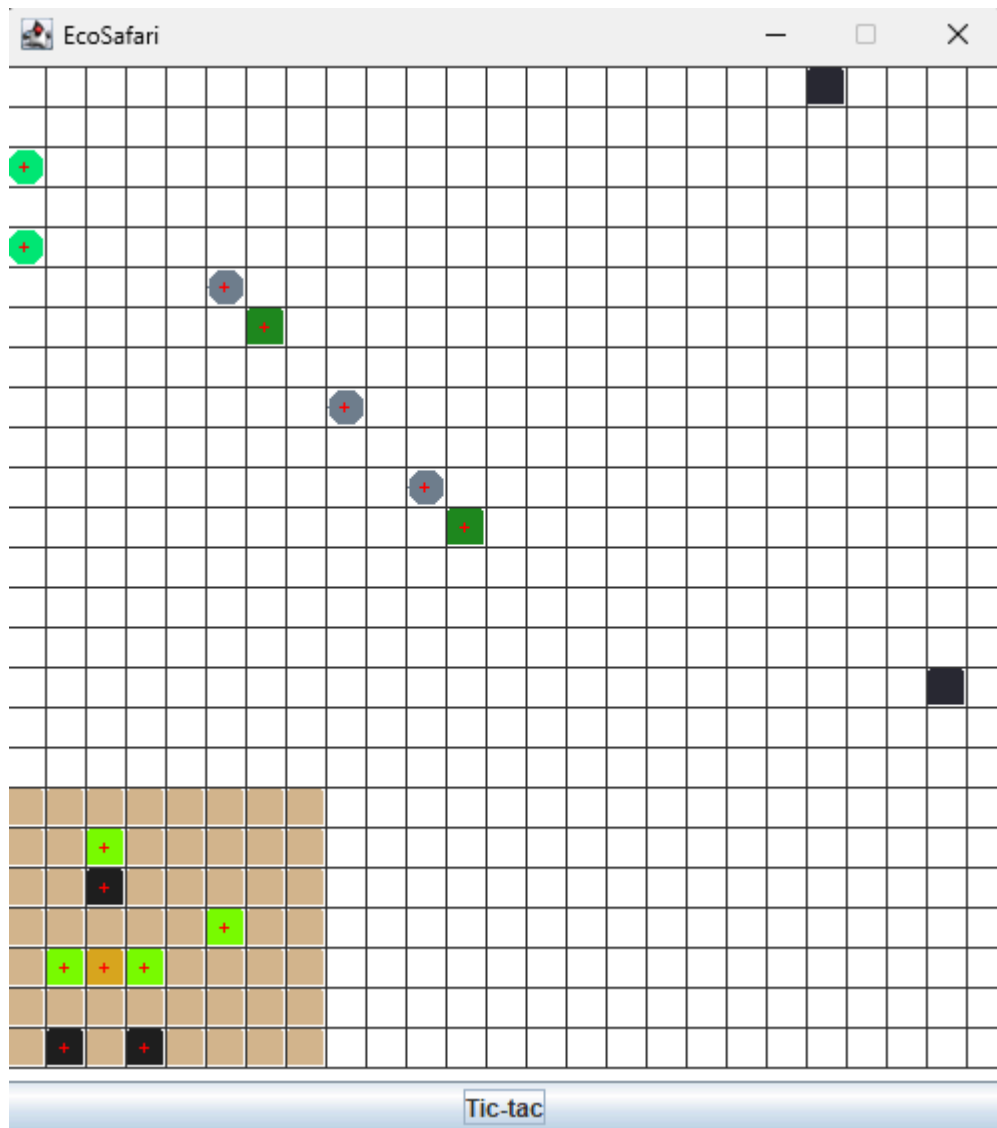
5 tics despues



Parte IV Empaquetando el proyecto

1. 

2. PS C:\Users\Cristian\DOPO_2026-2\Lab3> &C:\ProgramFiles\BlueJ\jdk\bin\java.exe"
-jar EcoSafariPresentation.jar
Se abrio la interfaz donde se puede presionar el Tic tac



3. Las ventajas que tiene el archivo es la practicidad a la hora de querer saber o probar la parte grafica e interactiva de el proyecto sin necesidad de entrar a el proyecto en BlueJ con riesgo a que un externo o por un error se lleguen a eliminar o modificar clases, que sean importantes dentro del proyecto. Además, sirve para independizar la parte gráfica de cierta forma, sin embargo toda la parte de domain la sigue alimentando y respaldando.

Retrospectiva

1. Tiempo total invertido en el laboratorio por cada integrante

El laboratorio demandó un total acumulado de 9 horas de trabajo colaborativo e individual, distribuidas equitativamente según las competencias y responsabilidades asumidas dentro del ciclo de desarrollo:

Integrante	Horas	Responsabilidades y Módulos Desarrollados
Cristian Salamanca	4 h	Estandarización de documentación técnica (Javadocs formales de clases y métodos, omitiendo atributos), diseño e implementación de la entidad Storm (Tormenta, Ciclo 3), diseño e implementación de la nueva entidad polimórfica Chameleon (Camaleón, Ciclo 5) y

		construcción de sus respectivas pruebas unitarias automatizadas.
Juan David Espitia	5 h	Arquitectura modular por paquetes (domain, presentation, test), diseño e implementación del modelo depredador-presa-recurso (Parte III: jerarquía Animal, Lion, Zebra, Ground, Grass), nuevo elefante SavannaElephant (Ciclo 4), arbustos Shrub (Ciclo 2), integración del motor discreto ticTac(), refactorización para eliminación de código duplicado y determinismo en JUnit.

2. Estado actual del laboratorio

Estado: Concluido y Verificado al 100%. El laboratorio se encuentra en estado 100% culminado, funcional y rigurosamente verificado. Se completaron a cabalidad todas las especificaciones teóricas y prácticas planteadas en la guía de trabajo:

- Cobertura Integral de Ciclos: Se desarrollaron exitosamente los Ciclos 1 al 5 y la Parte III (Reto de la Pradera: modelo depredador-presa-recurso). Cada entidad y componente (Elephant, Shrub, Storm, SavannaElephant, Chameleon, Lion, Zebra, Ground, Grass) cumple estrictamente los principios de herencia, encapsulamiento, abstracción y polimorfismo, evitando la redundancia estructural.
- Batería de Pruebas Unitarias Automatizadas: Se implementó un conjunto robusto de 29 pruebas unitarias automatizadas con JUnit 4.12 distribuidas en 8 suites de prueba independientes (ElephantTest, ShrubTest, StormTest, SavannaElephantTest, ChameleonTest, PrairieTest, PredationTest y EcoSafariExceptionTest). La totalidad de las 29 pruebas pasa de manera determinística (100% de éxito, 0 fallos).
- Arquitectura y Empaquetamiento: El código fuente se encuentra desacoplado en capas lógicas (domain, presentation y test), con manejo estandarizado de excepciones de dominio (EcoSafariException), documentación técnica Javadoc completa y preparado para su distribución como archivo ejecutable JAR.

3. Evaluación de prácticas Extreme Programming (XP): Test Unit First

Práctica Más Útil: Testing – Code the unit test first. Dentro del compendio de prácticas ágiles de Extreme Programming (XP) aplicadas a lo largo de este laboratorio, la práctica más determinante, esclarecedora y útil fue Testing: Code the unit test first (Test Unit First / Desarrollo Guiado por Pruebas).

En un sistema de simulación discreta bidimensional como EcoSafari, el comportamiento de las entidades no opera de forma aislada, sino bajo una red dinámica de interacciones concurrentes y reglas de estado. Escribir la prueba unitaria antes de formular el código de producción aportó ventajas metodológicas decisivas:

1. Especificación Contractual y Diseño Modular: Formular las pruebas unitarias en primer lugar obligó a definir con claridad las responsabilidades de cada clase, los invariantes biológicos (la energía no puede superar el 100% ni continuar viva con 0%) y los contratos de interfaz necesarios para evitar código redundante.
2. Guía para la Abstracción sin Duplicación: Al diseñar las pruebas para Lion y Zebra de forma previa, se evidenció de inmediato que compartían cinemática, intercambio de celdas

con tierra y consumo de energía, lo que impulsó la extracción temprana de esta lógica hacia una superclase abstracta común (`Animal`), impidiendo la duplicación de código desde la raíz.

3. Refactorización Continua Segura: Permitió reestructurar algoritmos críticos —como la parametrización simétrica en 4 ejes de `Ground.newAnimal()`— con la certeza matemática de no alterar funcionalidades previas ni introducir regresiones.

Como práctica complementaria, Coding Standards aseguró la uniformidad en nombres, visibilidad y documentación formal sin comentarios informales de línea única.

4. Mayor logro alcanzado y justificación técnica

Logro Principal: Modelado y equilibrio del ecosistema depredador-presa-recurso bajo un diseño libre de redundancias. El logro de mayor trascendencia técnica residió en el modelado, balance y sincronización del ecosistema depredador-presa-recurso (Parte III) mediante una arquitectura orientada por objetos limpia, reutilizable y polimórfica en `EcoSafari.ticTac()`.

Este aspecto constituyó el mayor hito técnico del proyecto al integrar armónicamente múltiples dinámicas concurrentes en una matriz bidimensional con vecindario de Moore de 8 celdas:

- Jerarquía trófica polimórfica: Se logró modelar la locomoción diferencial, el gasto energético por paso y la alimentación selectiva (león devora cebra sumando 50% de energía; cebr devora hierba sumando 25%) a través del despacho dinámico de métodos, manteniendo clases concretas compactas y altamente especializadas.
- Algoritmo de reproducción espacial colineal (`Ground.newAnimal()`): Se implementó la detección simétrica en los 4 ejes del plano discreto (horizontal $\{1, 0\}$, vertical $\{0, 1\}$, diagonal principal $\{1, 1\}$ y diagonal secundaria $\{1, -1\}$). Este diseño interpretó fielmente el requerimiento ("en la celda de tierra") y evitó una explosión demográfica exponencial descontrolada, manteniendo el balance ecológico y la estabilidad de la simulación.

5. Mayor problema técnico enfrentado y estrategia de resolución

Problema Técnico: Implementar correctamente los requerimientos del sistema sin incurrir en duplicación de código. El desafío técnico más exigente del proyecto radicó en implementar la totalidad de los requerimientos de comportamiento, movimiento, predación y reproducción del ecosistema cumpliendo estrictamente el principio DRY (Don't Repeat Yourself), evitando la duplicación de lógica transversal en las múltiples entidades del dominio.

- Naturaleza del Desafío: El laboratorio presentaba múltiples entidades con comportamientos parcialmente superpuestos. Por ejemplo: tanto Lion como Zebra debían desplazarse sobre celdas de tipo `Ground`, intercambiar posiciones, descontar un 10% de energía por movimiento y fallecer si su energía llegaba a 0%, diferenciándose únicamente en el avance (1 celda en el león frente a 2 celdas en la cebr) y en su objetivo alimenticio. Asimismo, la inspección de vecinos para predar y la detección simétrica de progenitores para la reproducción en `Ground` amenazaban con generar bloques masivos de condicionales redundantes e iteraciones repetidas si no se abordaban con abstracción adecuada.

- Estrategia de Resolución y Diseño:

1. Jerarquía Abstracta Animal (Patrón Template Method): Se factorizó la cinemática común en la clase abstracta Animal, parametrizando la cantidad de pasos a través de atributos protegidos e implementando el algoritmo general de desplazamiento e intercambio con tierra en un único método `moveAnimal()`. De este modo, las subclases Lion y Zebra solo implementan su lógica particular de alimentación (`eat()`) y reproducción (`reproduce()`), reutilizando íntegramente la lógica de locomoción y control de energía.

2. Vectorización Simétrica en `Ground.newAnimal()`: En lugar de replicar bloques condicionales para cada dirección cardinal y diagonal, se parametrizó la búsqueda en un vector de desplazamientos relativos (`{1, 0}`, `{0, 1}`, `{1, 1}`, `{1, -1}`). Un único bucle evalúa simétricamente los extremos colineales y delega la creación del nuevo individuo al método polimórfico `reproduce()` del animal encontrado, logrando cobertura total en los 4 ejes sin duplicar una sola línea de código.

3. Reutilización en Pruebas Unitarias: Esta misma separación limpia permitió aislar la cinemática de las entidades de los efectos secundarios estocásticos de la matriz (como el brote aleatorio de hierba en `Ground`), resolviendo con facilidad los tests unitarios y garantizando un 100% de determinismo.

6. Desempeño del equipo y compromisos de mejora

Evaluación del Desempeño: Aciertos Destacados del Equipo:

- Cohesión y Modularidad en el Reparto de Tareas: Se estructuró una división de trabajo que favoreció el diseño sin duplicaciones; mientras un integrante se enfocó en las entidades polimórficas complementarias (Storm, Chameleon) y en la estandarización de la documentación técnica formal, el otro abordó la arquitectura base de la pradera, la jerarquía Animal y el motor discreto `ticTac()`.

- Revisiones Cruzadas de Código (Code Review): Se realizaron inspecciones continuas del código fuente para asegurar la eliminación de código redundante, la correcta aplicación del polimorfismo y el estricto cumplimiento de los estándares de codificación solicitados.

Compromisos de Mejora: Compromisos de Mejora para Próximos Ciclos:

- Refactorización Temprana basada en Patrones: Continuar identificando oportunidades de abstracción (como patrones Strategy o State) desde la formulación inicial de diagramas UML en Astah, previniendo refactorizaciones tardías.

- Desacoplamiento de Componentes Estocásticos: En futuras simulaciones que involucren probabilidades aleatorias, se implementará inyección de dependencias para generadores pseudoaleatorios configurables con semilla fija, facilitando pruebas determinísticas a nivel de integración completa.